

Session 5: Subagents, Hooks, and Final Project

Spawning focused agents, event-driven automation, and an end-to-end research artifact

Brady Allardice

UPF & IBEI

A focused agent spawned for a bounded job —
the right tool for the parallelizable, scoped work
that already shows up in your pipeline.

Concrete patterns, then named teams.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary to your main conversation — not its raw trace

What that buys you:

- ➊ **Parallelism** — run three searches at once instead of one after the other
- ➋ **Context hygiene** — a big messy exploration stays out of your main context budget

Think of it as

Delegating a bounded sub-task to a focused colleague who returns a clean, structured answer — so your main conversation stays on the high-level work.

Where Subagents Shine

Reach for a subagent when the subtask is:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without reading the full trace

Concrete research use cases:

- **Multi-database literature searches** — one agent per database, all running in parallel
- **Coding many documents** against a fixed rubric
- **Parallel robustness checks** across independent specifications
- **Running the same sanity-check** script over many datasets
- **Exploring a codebase** — send a subagent to map out the files, keep the summary in your main context

Three habits that make subagents pay off:

- ❶ **Brief them like a contractor.** Concrete deliverable, all the context they need up front — they don't see your conversation history.
- ❷ **Ask for structured output** (bullet list, table, JSON). You'll be consuming the result; make it easy to read and verify.
- ❸ **Verify independently.** You get a summary, not the full trace. Spot-check the way you would a student's work.

The payoff

A literature sweep, robustness battery, or repo survey that would have taken an hour finishes in minutes — and doesn't eat into your main conversation's context. That's what makes them worth reaching for.

Building a Team of Specialized Agents

Named subagents you define once and invoke on demand. Each has its own scope, system prompt, and tool allowlist — like a team of focused collaborators.

Define one: a markdown file at `~/.claude/agents/<name>.md` (user-level) or `.claude/agents/<name>.md` (project-level).

```
---
name: lit-search
description: Use for multi-database literature searches on a topic.
tools: [mcp__zotero__*, WebSearch, WebFetch]
model: sonnet
---

You are a literature search specialist. Return a structured
bibliography with the key claim from each paper.
```

Activate it:

- Claude reads every agent's description at startup and spawns one when a task matches
- Or invoke it explicitly: *"use the lit-search agent to..."*
- Spawn several in a single message — they run in parallel

Permissions work as usual: any tools the agent uses need to be allowed in `.claude/settings.json`.